TP - Matrix Multiplication: Thread Mappings and Memory Access

Oguz Kaya
oguz.kaya@universite-paris-saclay.fr

Part 1

The exercise: one product, five kernels

In this lab you will compute the product $C = AB$ of two $N \times N$ single-precision matrices on the GPU, five times. What changes from one kernel to the next is only *which thread computes which element of* C. The goal is the same as last week's: decide, for every kernel, whether the memory accesses of a warp are **coalesced** – and this time, to see what it costs when they are not.

The matrices are allocated as one-dimensional arrays of $N \cdot N$ floats (`malloc` on the CPU, `cudaMalloc` on the GPU), and their layouts are *imposed*:

- A is **row-major**: $A(i, k)$ is at `A[i * N + k]`;
- B is **column-major**: $B(k, j)$ is at `B[k + j * N]`;
- C is row-major: $C(i, j)$ is at `C[i * N + j]`,

and $C(i, j) = \sum_k A(i, k)\, B(k, j)$: a thread that computes $C(i, j)$ walks along row i of A and down column j of B. Both walks are contiguous in memory for *that thread*; the question is what the 32 threads of a warp touch *together* at each step k. N is any positive integer: every kernel must round its grid up and guard the threads that fall outside the matrix.

You will work in the provided skeleton code **matmul.cu**. It contains the five kernels with a comment block above each one describing the mapping to implement, and a `main()` that launches every kernel, checks its result against a CPU reference and prints its running time and its performance in GFLOP/s. Compile and run it with

```
nvcc -O2 -arch=native matmul.cu -o matmul -lcublas
./matmul
```

or paste it into `cuda.godbolt.org`, which runs it on a Tesla T4. `matmul.cu` calls cuBLAS, so it *must* be linked with `-lcublas`: on godbolt, the compiler options of the pane that runs the program must read `-O2 -lcublas`; without `-lcublas` the program does not link. The matrix size is the constant `n` at the top of `main()` (250 by default: not a multiple of 32 nor of 256, so that the guards are tested); change it there and rerun. As given, the program compiles and runs, and every kernel prints `WRONG` until you write it.

For each kernel you have to write its body following the mapping described in its comment block, set its `dim3` grid and block dimensions in `main()`, and, for kernels 2–5, write in the comment block, under `ANALYSIS`, what the 32 threads of a warp touch in memory at one instruction – for the read of A and the read of B at a step k, and for the write of C – with the conclusion: coalesced, or not. Remember that a warp is 32 consecutive values of `threadIdx.x` (with the same `threadIdx.y`), that the elements of a row of A are contiguous and those of a column of B are contiguous, and that a 32-byte sector holds 8 floats: 32 consecutive floats are 4 sectors, 32 floats that are $4N$ bytes apart are 32 sectors. The analysis is the point of the lab; the timings will confirm or surprise.

The pictures next to each part show what one block computes (dark cells of C) and what it reads (bands of A and B); they use $N = 14$ and blocks of 4 threads, the code uses blocks of 256 and 32×32.

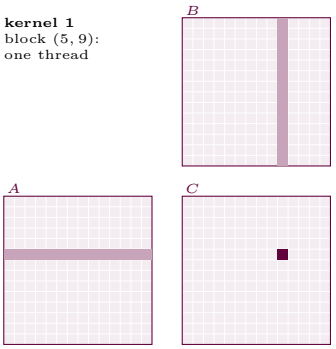
Part 2

One thread per block

A 2D grid of $N \times N$ blocks of a single thread, block (i, j) computing the element $C(i, j)$ from row i of A and column j of B. With one thread per block the question of coalescing does not arise: no analysis for this one.

Ex. 1

a) Write `matmul_blocks` (kernel 1) and its launch dimensions; compile, run, and check that you understand every column of the output.



block $(5, 9)$ computes $C(5, 9)$

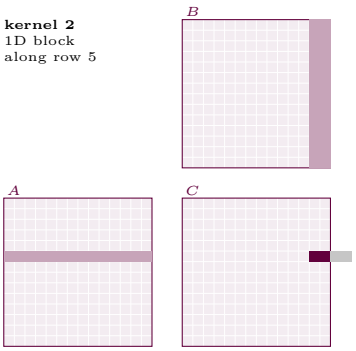
Part 3

1D blocks along a row of C

Each block has `BS_1D` $= 256$ threads and computes 256 consecutive elements of *one row* of C, one element per thread; the grid covers every row and every piece of a row. The last block of a row sticks out of the matrix when N is not a multiple of 256: its threads must be guarded.

Ex. 2

a) Write `matmul_1d_row` (kernel 2), its launch dimensions, and its analysis.



a block of 4 threads on row 5; grey: threads outside the matrix

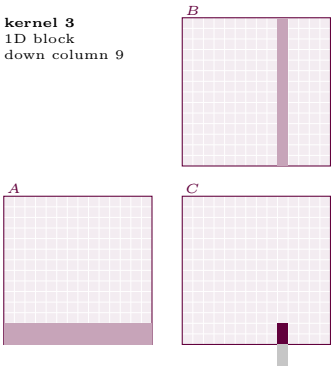
Part 4

1D blocks along a column of C

The same blocks of 256 threads, each computing 256 consecutive elements of *one column* of C; the grid covers every column and every piece of a column.

Ex. 3

a) Write `matmul_1d_column` (kernel 3), its launch dimensions, and its analysis.



a block of 4 threads on column 9

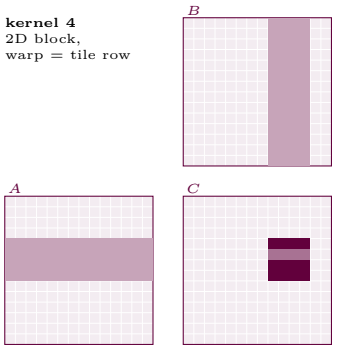
Part 5

2D blocks, warps horizontal

Each block is now `BS_2D` × `BS_2D` = 32 × 32 threads and computes a 32 × 32 tile of C, one element per thread, with `threadIdx.x` running *along the row* of C: a warp is one row of the tile. The grid covers the matrix with tiles; the tiles of the last row and of the last column stick out when N is not a multiple of 32.

Ex. 4

a) Write `matmul_2d_horizontal` (kernel 4), its launch dimensions, and its analysis.



a 4 × 4 block; a warp (lighter) is a row of the tile

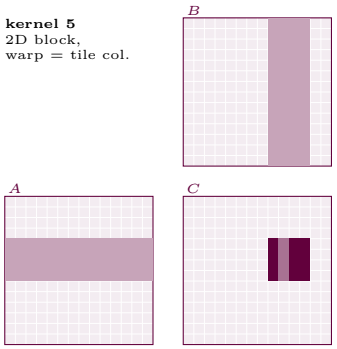
Part 6

2D blocks, warps vertical

The same 32 × 32 tile of C per block, with `threadIdx.x` running *down the column* of C: a warp is one column of the tile.

Ex. 5

a) Write `matmul_2d_vertical` (kernel 5), its launch dimensions, and its analysis.



the same tile; a warp (lighter) is a column of the tile

Part 7

cuBLAS

`cublasSgemm` is NVIDIA's single-precision matrix product. cuBLAS follows the BLAS convention: every matrix is *column-major*, and

```
cublasSgemm(handle, opX, opY, m, n, k, &alpha, X, ldx, Y, ldy, &beta, C, ldc)
```

computes $C \leftarrow \alpha\,\mathrm{op}(X)\,\mathrm{op}(Y) + \beta\,C$, with op either `CUBLAS_OP_N` (the matrix as stored) or `CUBLAS_OP_T` (its transpose), $\mathrm{op}(X)$ of size $m \times k$, $\mathrm{op}(Y)$ of size $k \times n$, and `ld*` the leading dimensions (the distance, in elements, between two consecutive columns). Our A and C are row-major, which cuBLAS cannot be told; think about what it sees when it reads our arrays as column-major matrices, and which product leaves $C = AB$ in `dC` in *our* layout.

Ex. 6

a) Write the `cublasSgemm` call in `cublasMatmul` (kernel 6). The driver checks its result like any other kernel.

b) Compare the GFLOP/s of your kernels with those of cuBLAS for `n` = 128, 256, 512, 1024, 2048 and 4096.